

CHAPITRE VIII - LES PILES

1. Définition

La pile est une structure de données abstraite linéaire (chaque elt a un successeur et un prédécesseur sauf le premier et le dernier) dont le seul et unique élément accessible est le sommet de la pile. Elle est appelée aussi LIFO (Last in First Out) cad dernier-entré-premier-sorti. Les opérations d'insertion/ suppression ne se font qu'au sommet.

Exemple d'utilisation de la pile

L'appel en cascade de plusieurs sous-programmes est géré par une pile. Si le programme A appelle B qui appelle C qui appelle D, la pile des programmes en attente évoluera comme suit :

				C			
			B	B	B		
Pile		A	A	A	A	A	
Temps	T0	T1	T2	T3	T4	T5	T6

T0 : lancement du programme A

T1 : appel du programme B par A

T2 : appel du programme C par B

T3 : appel du programme D par C

T4 : poursuite du programme C (quand D est terminé)

T5 : poursuite du programme B (quand C est terminé)

T6 : poursuite du programme A (quand B est terminé)

2. Type abstrait « Pile »

Les opérations sur les piles sont :

Créer une pile ; Tester si une pile est vide ; accéder au sommet d'une pile ; empiler un élément ; retirer l'élément qui se trouve au sommet (dépiler).

La signature du type est donc :

Sorte : Pile

Utilise booléen, Elément

Opérations :

Pile-Vide : $\rightarrow$ Pile

Empiler : Pile x Element $\rightarrow$ Pile

Dépiler : Pile $\rightarrow$ Pile

Sommet : Pile $\rightarrow$ Element

Est-vide : Pile $\rightarrow$ Booléen

Pré-conditions :

Dépiler (P) **est défini ssi** Est-vide (P) = faux

Sommet (P) **est défini ssi** Est-vide (P) = faux

Axiomes :

P est une Pile, et e est un Element

Dépiler (Empiler (P,e))=P

Sommet (Empiler (P,e)) =e

Est-vide (Pile-Vide)=vrai

Est-vide (Empiler (P,e))=faux

Pour implémenter les piles on peut utiliser les représentations étudiées pour les listes.

3. Représentation des Piles

a. Représentation contigüe

P est représenté par un tableau contenant les éléments de la pile et un indice indiquant la position du sommet.

Avantages et inconvénients :

Manipulation très facile de la pile et non coûteuse en temps ; mais nécessité de majorer le nombre d'éléments de la pile !

b. Représentation chaînée

Les éléments de la pile sont chaînés entre eux, et le sommet d'une pile non vide est le premier de la liste ; la pile vide est représentée par Nil.

Développer les opérations empiler et depiler.

4. Exercice

Pour passer d'un nombre en base 10 à un nombre en base N , on peut appliquer la méthode suivante :

Soit K le nombre en base 10 à convertir en base N .

1. Effectuer la division entière de K par N . Soit D le résultat de cette division et R son reste ;
2. Si $D > 0$, revenir à l'étape 1 en remplaçant K par D .
3. Sinon, l'écriture en base N de K est égale à la concaténation de tous les restes obtenus en commençant par le dernier.

Ecrire une procédure qui, étant donnés les deux nombres strictement positifs K et N , permet de convertir K en base N et d'afficher le résultat. Utiliser une pile comme structure intermédiaire.

CHAPITRE IX - LES FILES

1. Définition

Une file est une liste d'éléments caractérisée par deux extrémités (une tête et une queue). Dans le cas d'une file, les insertions se font à la queue et les suppressions et accès se font à la tête. Par analogie avec les files d'attente on dit que l'élément présent depuis le plus longtemps est le premier. Les files d'attente sont aussi appelées FIFO (First In First Out) c a d premier entré premier sorti.

2. Type abstrait de données « File »

Les opérations sur les files sont :

Tester si la file est vide ; accéder au premier élément de la file ; ajouter un élément dans la file; retirer le premier élément de la file.

La signature du type File est donc :

Sorte : File

Utilise booléen, Elément

Opérations :

File-Vide : $\rightarrow$ File

Enfiler : File x Elément $\rightarrow$ File #ajouter

Défiler : File $\rightarrow$ File # retirer

Premier : File $\rightarrow$ Elément

Est-vide : File $\rightarrow$ Booléen

Pré-conditions :

Defiler (F) **est défini ssi** est-vide (F) = faux

Premier (F) **est défini ssi** est-vide (F) = faux

Axiomes :

Les axiomes diffèrent de ceux de la pile en ce qui concerne les opérations Premier et Retirer.

Pour tout F de sorte File et e de sorte Elément, on a :

$\text{Est-vide}(F) = \text{vrai} \Rightarrow \text{Premier}(\text{Enfiler}(F, e)) = e$

$\text{Est-vide}(F) = \text{faux} \Rightarrow \text{Premier}(\text{Enfiler}(F, e)) = \text{Premier}(F)$

$\text{Est-vide}(F) = \text{vrai} \Rightarrow \text{Defiler}(\text{Enfiler}(F, e)) = \text{File-vide}$

$\text{Est-vide}(F) = \text{faux} \Rightarrow \text{Defiler}(\text{Enfiler}(F, e)) = \text{Enfiler}(\text{Defiler}(F), e)$

$\text{Est-vide}(\text{File-vide}) = \text{vrai}$

$\text{Est-vide}(\text{Enfiler}((F, e))) = \text{faux}$

On peut représenter les files de manière contigüe ou de manière chaînée.

3. Représentation des files

a. Représentation contigüe

Exemples :

Développer les opérations enfiler et defiler.

b. Représentation chaînée

Développer les opérations enfiler et defiler.

4. Exercice

Ecrire un algorithme qui permet de fusionner deux files F1 et F2 à éléments ordonnés dans une file F3 ordonnée.

CHAPITRE X - LES ARBRES BINAIRES

1. Définition

Une **arborescence** désigne la représentation d'une structure hiérarchique à plusieurs niveaux, rappelant la structure d'un arbre. Une structure arborescente est un ensemble d'informations homogènes organisées en niveaux : chaque information d'un niveau peut être reliée à plusieurs informations du niveau supérieur.

En informatique, **l'organisation des fichiers sur un support de stockage est structurée en arborescence**, chaque répertoire ou dossier étant une *branche* pouvant comporter des *feuilles* (fichiers) et des nœuds de départ d'autres branches (sous-dossiers). À la base d'une arborescence se trouve un répertoire appelé *racine*. Ce répertoire peut contenir des fichiers et des répertoires, qui eux-mêmes peuvent contenir la même chose. **C'est aussi sous forme d'arbre que sont organisés les fichiers dans des systèmes d'exploitation tels qu'UNIX. Les programmes traités par un compilateur** sont représentés sous forme d'arbre.

Une propriété intrinsèque de la structure d'arbre est la récursivité. Les définitions des caractéristiques des arbres, aussi bien que les algorithmes qui les manipulent s'écrivent très naturellement de manière récursive.

2. Terminologie

Dans ce qui suit nous utiliserons un vocabulaire inspiré des arbres généalogiques.

Racine : nœud de niveau zéro (n'a pas de père).

Nœud : racine (ou sommet) de tout sous-arbre. Chaque nœud a 0 ou plusieurs fils.

Feuille : un nœud qui n'a pas de fils.

Fils : x est le fils de y si x est la racine du sous arbre y.

Père : x est le père de y si y est le fils de x.

Frère : deux nœuds sont frères s'ils ont le même père.

Ascendants : A est ascendant de B si A est le père de B ou un ascendant de B.

Descendants : X est descendant de Y si X est le fils de Y ou un descendant d'un fils de Y.

Branche : tout chemin de la racine à une feuille de l'arbre.

Hauteur d'un nœud (ou profondeur ou niveau) est définie récursivement comme suit : étant donné x un nœud d'un arbre B,

$h(x) = 0$ si x est la racine de B et, $h(x) = 1 + h(y)$ si y est le père de x.

Avantages de l'utilisation des structures arborescentes :

- Adaptation à la représentation naturelle des informations homogènes organisées ;
- Grande commodité et rapidité de manipulation des informations grâce à la notion de récursivité.

3. Définition du TAD

Le type arbre binaire est soit vide (noté $\emptyset$), soit de la forme $B = \langle o, B1, B2 \rangle$, où $B1$ et $B2$ sont des arbres binaires disjoints et 'o' est un nœud appelé racine. $B1$ est le sous-arbre gauche de la racine de B et $B2$ est sous-arbre droit.

Sorte arbre

Utilise : **Nœud, Element**

Opérations

Arbre-vide : $\rightarrow$ Arbre

$\langle -, -, - \rangle$: Nœud $\times$ Arbre $\times$ Arbre $\rightarrow$ Arbre

Racine : Arbre $\rightarrow$ Nœud

g : Arbre $\rightarrow$ Arbre

d : Arbre $\rightarrow$ Arbre

contenu : Nœud $\rightarrow$ Element

Préconditions

racine($B1$) est défini Ssi $B1 \neq$ Arbre-vide

g($B1$) est défini Ssi $B1 \neq$ Arbre-vide

d($B1$) est défini Ssi $B1 \neq$ Arbre-vide

Axiomes

racine($\langle o, B1, B2 \rangle$) = o

g($\langle o, B1, B2 \rangle$) = $B1$

d($\langle o, B1, B2 \rangle$) = $B2$

4. Mesures sur les arbres

- La **taille** d'un arbre est le nombre de ses nœuds ; on définit récursivement l'opération taille par :

taille(arbre-vide) = 0 et

taille($\langle o, B1, B2 \rangle$) = 1 + taille($B1$) + taille($B2$).

- La **hauteur d'un nœud** (on dit aussi profondeur ou niveau) est définie récursivement de la façon suivante, étant donné x un nœud de B,

$h(x) = 0$ si x est racine de B, et $h(x) = 1 + h(y)$ si y est le père de x.

- La **hauteur ou profondeur d'un arbre B** est :

$h(B) = \max \{h(x) ; x \text{ nœud de } B\}$

5. Arbres binaires particuliers

- Un arbre binaire est **dégénéré** ou **filiforme** si tous ses nœuds ont au maximum un seul fils.
- Un arbre binaire est **complet** s'il contient un nœud au niveau 0, 2 nœuds au niveau 1, 4 nœuds au niveau 2. D'une façon générale **2^h nœuds au niveau h**. (*chaque nœud est complètement rempli*)
- Un **arbre binaire parfait** est tel que tous les niveaux sauf éventuellement le dernier sont remplis, et dans ce cas les feuilles du dernier niveau sont groupées à gauche.

6. Représentation des arbres binaires

a. Représentation chaînée

Nœud = structure

val : Element

g, d : ^Nœud

Fin

Type Arbre = ^Nœud

A : Arbre

$A = \text{Nil} \Leftrightarrow \text{Arbre vide}$, $A.^{\text{val}} \Leftrightarrow \text{contenu}(\text{Racine}(A))$, $A.^{\text{g}} \Leftrightarrow \text{g}(A)$, $A.^{\text{d}} \Leftrightarrow \text{d}(A)$

b. Représentation contigüe

Avec cette représentation le nombre de nœuds de l'arbre est limité et ne doit pas dépasser une valeur Max.

i. Représentation indexée

A chaque nœud de l'arbre on associe une valeur ainsi que les indices des deux fils gauche et droit.

On prendra la convention suivante : arbre vide $\Leftrightarrow$ indice = 0.

Nœud = struct

Val : Element

g, d : 0..Max

fin struct

Type ArbTab = tableau [1..Max] de Nœud

Arbre = struct

Rac : 0..Max

T : ArbTab

Fin struct

Exemple :

Un exemple d'arbre binaire serait représenté par une variable A de type Arbre, dont la racine A.Rac contient l'indice 3 et le champ T contient le tableau suivant.

V	G	D
d	0	10
a	5	6
g	0	0
b	2	0
c	13	11
f	0	0
m	0	0
e	8	4
l	9	0
k	0	0

Traduction des opérations définies dans le TAD.

$A.Rac=0 \Leftrightarrow A = \text{arbre-vide}$,

Si $A.Rac=r$ et $r>0$ alors l'arbre n'est pas vide. $A.T[Rac]=\text{racine}(A)$, $A.T[Rac].val=\text{contenu}(\text{racine}(A))$ et $A.T[Rac].G$ et $A.T[Rac].D$ sont les indices de $g(A)$ et $d(A)$.

Remarque

Dans cette représentation indexée, l'indice associé à chaque nœud est arbitraire ; la suppression/ajout d'un nœud ne présente donc pas de difficultés dans la limite des places réservées pour le tableau. On garde donc un des avantages de l'allocation dynamique.

ii. Représentation séquentielle

Cette représentation se base sur la convention suivante : si un nœud est numéroté par i , son fils gauche se trouve à l'indice $2i$ et son fils droit se trouve à l'indice $2i+1$. Dans cette représentation le passage d'un nœud à un autre se traduit par un simple calcul d'indice dans le tableau :

$2 \leq i \leq \text{Max} \Rightarrow$ le père du nœud d'indice i est à l'indice $i \text{ div } 2$

$1 \leq i \leq \text{Max div } 2 \Rightarrow$ le fils gauche du nœud d'indice i est en $2i$

Le fils droit du nœud d'indice i est en $2i+1$

Notons que cette représentation nous impose, dans le cas d'arbres binaires quelconques, de laisser des cases vides dans le tableau pour marquer la place des nœuds non présents. Ceci nous fait perdre l'avantage de compacité du tableau.

7. Parcours d'un arbre binaire

Parcourir un arbre revient à visiter tous ses nœuds. Une des opérations les plus fréquentes mises en œuvre par les algorithmes qui manipulent les arbres consiste à parcourir ceux-ci. Il existe plusieurs ordres dans lesquels les nœuds peuvent être visités, et chacun a des propriétés utiles qui peuvent être exploitées par les algorithmes basés sur les arbres binaires.

a. Parcours en largeur

Ce parcours essaie toujours de visiter le nœud le plus proche de la racine qui n'a pas été visité. En suivant ce parcours, on va d'abord visiter la racine, puis les nœuds à la profondeur 1, puis 2... D'où le nom parcours en largeur.

b. Parcours en profondeur main gauche

Ce parcours consiste à tourner autour de l'arbre en suivant le chemin indiqué sur l'exemple. Ce chemin part à gauche de la racine, et va toujours le plus à gauche possible en suivant l'arbre.

Si on essaie d'écrire un algorithme récursif qui reflète ce parcours, on va visiter 3 fois chaque nœud. Appelons Traitement1 (resp. Traitement 2 et Traitement 3) la suite d'actions exécutées lorsqu'un nœud est rencontré pour la première (resp. deuxième et troisième) fois. L'algorithme est le suivant :

Procédure Parcours_P (DON A : Arbre)

Debut

Si ($A \neq \text{Arbre_vide}$) alors

Traitement 1 (A)	#première visite
Parcours_P(g(A))	
Traitement 2 (A)	# 2eme visite
Parcours_P(d(A))	
Traitement 3	# 3 ^{ème} visite

Fsi

Fin

On remarque ici que la complexité est assez élevée puisque chaque nœud est visité 3 fois. Souvent on n'a pas 3 traitements différents à exécuter : par exemple si on veut écrire les valeurs des nœuds de l'arbre en appliquant cet algorithme on va les écrire 3 fois et c'est inutile.

D'où l'intérêt de l'existence de trois ordres classiques d'exploration de l'arbre :

- Ordre préfixé : père, fils gauche, fils droit
- Ordre infixé : fils gauche, père, fils droit
- Ordre postfixé : fils gauche, fils droit, père

c. Exercices

1. Ecrire une procédure récursive qui permet d'afficher les valeurs des nœuds d'un arbre binaire selon l'ordre préfixé chaque valeur est affichée une et une seule fois.
2. Ecrire une version itérative de la procédure précédente.

CHAPITRE XI - LES ARBRES BINAIRES DE RECHERCHE

1. Définition

Un arbre binaire de recherche est un arbre binaire tel que pour tout nœud v de l'arbre :

- Les éléments de tous les nœuds du sous arbre gauche de v sont inférieurs ou égaux à l'élément contenu dans v .
- Les éléments de tous les nœuds du sous arbre droit de v sont supérieurs à l'élément contenu dans v .

Il en résulte de cette définition que le parcours symétrique d'un arbre binaire de recherche produit la suite des éléments triée en ordre croissant.

Avantages : recherche et tri efficaces

2. Recherche d'un élément dans un arbre binaire de recherche

Pour rechercher une occurrence d'un élément dans un arbre binaire de recherche, on compare cet élément au contenu de la racine :

- S'il y a égalité, l'élément est trouvé et la recherche s'arrête ;
- Si l'élément est plus petit (resp. plus grand) que celui de la racine, on poursuit la recherche dans le sous-arbre gauche (resp. droit) ; si ce sous-arbre est vide, il y a échec.

3. Adjonction d'un élément

a. Adjonction aux feuilles

Principe

On compare l'élément au contenu de la racine pour savoir si l'ajout doit être fait dans le sous-arbre gauche ou dans le sous-arbre droit. On fait un appel récursif jusqu'à arriver à un arbre vide. On insère à cet emplacement l'élément à ajouter.

Procédures

b. Adjonction à la racine**Principe :**

L'adjonction à la racine peut être utile si les recherches portent sur les éléments **récemment ajoutés**.

Pour ajouter un élément X à la racine d'un arbre binaire de recherche, il faut d'abord couper l'arbre binaire de recherche en deux arbres binaires de recherche G et D contenant respectivement tous les éléments inférieurs ou égaux à X, et tous les éléments supérieurs à X, puis former l'arbre dont la racine contient X et qui a pour sous-arbre gauche (resp. droit) l'arbre G (resp. D). On obtient bien un arbre binaire de recherche.

L'étape de coupure est la plus délicate ; il est important de remarquer qu'il n'est pas nécessaire de parcourir tous les nœuds de l'arbre initial A pour former G et D. Ce sont les nœuds situés sur le chemin C suivi lors de la recherche de X qui déterminent la coupure : en effet si un nœud de C contient un élément plus petit que X, il vient se placer sur le bord droit de G et par la propriété des arbres binaires de recherche, il entraîne avec lui dans G tout son sous arbre gauche. Si un nœud de C contient un élément plus grand que X, il vient se placer sur le bord gauche de D, et par la propriété des arbres binaires de recherche, il entraîne avec lui dans D tout son sous arbre droit

4. Suppression d'un élément

Pour supprimer un élément, il faut tout d'abord déterminer sa place, puis effectuer la suppression proprement dite, qui s'accompagne éventuellement d'une réorganisation des éléments.

Principe :

- Si le nœud à supprimer est sans fils : la suppression est immédiate ;
- Si le nœud possède un seul fils : il suffit de le remplacer par ce fils ;
- Si le nœud possède deux fils : il y a deux solutions
 - Remplacer l'élément à supprimer par l'élément immédiatement inférieur (le plus grand élément de son sous-arbre gauche)
 - Remplacer l'élément à supprimer par l'élément immédiatement supérieur (le plus petit élément de son sous-arbre droit)

Ces deux solutions sont équivalentes lorsque tous les éléments de l'arbre binaire de recherche sont distincts.

On utilisera dans ce qui suit la première solution.

La procédure **supprimearb** utilise la procédure **supmax** de suppression du maximum dans un arbre binaire de recherche.

La procédure **supmax** supprime le nœud contenant l'élément maximal dans un arbre binaire de recherche A non vide ; il donne pour résultats l'élément maximal et l'arbre binaire de recherche A privé du nœud contenant cet élément.

